SQL

CHEAT SHEETS

A Complete Reference for SQL

Querying · Joins · Aggregates · Window Functions · CTEs · Subqueries · Data Modification · Schema & Constraints · Transactions · Indexes

22 REFERENCE SHEETS

CONTENTS

Covers SQL · Joins · Aggregates · Window Functions · CTEs · Stored Procedures (2025)

Begin → Sheet 01

SQL Cheat Sheets

Table of Contents

22 reference sheets · syntax · examples · notes

SQL · Sheet 1

SQL Statement Order

SELECT · FROM · JOIN · WHERE · GROUP BY · HAVING · ORDER BY · LIMIT

DESCRIPTION

A SQL `SELECT` statement is built from a series of clauses, each with a specific purpose. While you write them in a fixed order top-to-bottom, the database engine executes them in a completely different sequence — understanding this distinction explains why certain operations (like filtering on an aggregate in a `WHERE` clause) are not allowed. Every query must have at least a `SELECT` and a `FROM`; all remaining clauses are optional and must appear in the order shown.

NOTES

- `FROM` runs first — the database needs to know the source before anything else
- `SELECT` runs 6th — aliases defined here are not available in `WHERE` or `HAVING`
- `WHERE` filters rows before grouping; `HAVING` filters after — they are not interchangeable
- Clauses shown in `[ ]` are optional; omit them when not needed

EXAMPLE — ALL CLAUSES USED

```
SELECT   department, COUNT(*) AS employee_count
FROM     employees AS e
JOIN     departments AS d ON e.dept_id = d.id
WHERE    e.status = 'active'
GROUP BY department
HAVING   COUNT(*) > 5
ORDER BY employee_count DESC
LIMIT    10
```

CODE EXPLAINED

- → `SELECT` — return dept name and a row count
- → `FROM` — main table is `employees`, aliased `e`
- → `JOIN` — link to `departments` on matching id
- → `WHERE` — only include active employees
- → `GROUP BY` — collapse rows into one per department
- → `HAVING` — drop departments with 5 or fewer staff
- → `ORDER BY` — largest departments first
- → `LIMIT` — return only the top 10 results

CLAUSE ORDER (WRITE ORDER)

CLAUSE	DESCRIPTION
`SELECT`	What to return
`FROM`	Main table / alias
`[JOIN]`	Link related tables
`[WHERE]`	Filter rows
`[GROUP BY]`	Collapse into groups
`[HAVING]`	Filter groups
`[ORDER BY]`	Sort results
`[LIMIT]`	Cap row count

EXECUTION ORDER

STEP	CLAUSE
1	FROM
2	JOIN
3	WHERE
4	GROUP BY
4b	HAVING
5	ORDER BY
6	SELECT
7	LIMIT

WHAT'S ALLOWED WHERE?

	WHERE	HAVING	ORDER BY
Aggregate	✗	✓	✓
Col alias	✗	✗	✓
Subquery	✓	✓	✗
Non-grouped	✓	✗	✗

⚠ COMMON MISTAKES

- Using an aggregate in `WHERE` — use `HAVING` instead
- Referencing a `SELECT` alias in `WHERE` or `HAVING` — it doesn't exist yet
- Forgetting `GROUP BY` when mixing aggregate and non-aggregate columns in `SELECT`
- Using `HAVING` without `GROUP BY` — allowed but almost always a logic error

✓ TIPS & BEST PRACTICES

- Write clauses in order — it's easier to read and maintain
- Use table aliases early in `FROM` and apply them consistently throughout
- Test `WHERE` conditions with a plain `SELECT *` before adding aggregates
- Add `LIMIT` during development to avoid accidentally fetching millions of rows

QUICK SUMMARY

- Write order ≠ execution order — know the difference
- Only `SELECT` and `FROM` are required
- `WHERE` → rows · `HAVING` → groups
- `SELECT` runs 6th — aliases aren't available until then

SELECT

DISTINCT · aliases · expressions · SELECT * · aggregates · subqueries · table qualification

DESCRIPTION

The `SELECT` clause defines what data is returned by a query. It can return raw column values, calculated expressions, aggregated data, or completely static values. Despite being written first in every query, `SELECT` is executed sixth — meaning aliases defined here are not available in `WHERE` or `HAVING`, only in `ORDER BY`. When mixing aggregated and non-aggregated columns, every non-aggregated column must appear in `GROUP BY`.

NOTES

- `SELECT *` returns all columns — fine for exploration, avoid in production
- `DISTINCT` applies to the entire row, not a single column
- Aliases defined in `SELECT` are available in `ORDER BY` but not `WHERE` or `HAVING`
- Use `t.col` table qualification when joining tables that share column names

EXAMPLE

```sql
SELECT DISTINCT
    e.department,
    e.first_name || ' ' || e.last_name  AS full_name,
    salary * 1.1                        AS projected_salary,
    COUNT(*)                            AS total,
    AVG(salary)                         AS avg_salary,
    (SELECT MAX(salary) FROM employees) AS company_max
FROM   employees AS e
GROUP BY e.department, e.first_name, e.last_name, e.salary
ORDER BY avg_salary DESC
```

CODE EXPLAINED

- → `DISTINCT` — removes duplicate rows from result
- → `e.department` — qualified with table alias
- → `||` — string concatenation (PostgreSQL / SQLite)
- → `AS full_name` — column alias for output
- → `salary * 1.1` — calculated expression
- → `COUNT(*)` — aggregate; requires GROUP BY
- → `(SELECT MAX...)` — scalar subquery inline

RETURN TYPES

SYNTAX	DESCRIPTION
`SELECT *`	All columns
`SELECT col1, col2`	Specific columns
`SELECT col AS alias`	Column with custom name
`SELECT DISTINCT col`	Unique values only
`SELECT 'static'`	Static / literal value
`SELECT NULL`	Null value
`SELECT (SELECT ...)`	Scalar subquery

EXPRESSIONS

SYNTAX	DESCRIPTION
`col * 1.1`	Arithmetic on column
`col1 || col2`	Concatenation — PostgreSQL
`CONCAT(c1, c2)`	Concatenation — MySQL
`CASE WHEN ... END`	Conditional expression
`COALESCE(col, 0)`	Null fallback
`CAST(col AS type)`	Type conversion

AGGREGATE FUNCTIONS

FUNCTION	DESCRIPTION
`COUNT(*)`	All rows inc. nulls
`COUNT(col)`	Non-null values
`COUNT(DISTINCT col)`	Unique non-null values
`SUM(col)`	Total
`AVG(col)`	Average
`MIN(col)`	Smallest value
`MAX(col)`	Largest value

⚠ COMMON MISTAKES

- Using a `SELECT` alias in `WHERE` — it doesn't exist at that execution step
- Mixing aggregate and non-aggregate columns without `GROUP BY`
- Assuming `DISTINCT` applies to just one column — it applies to the whole row
- Using `SELECT *` in production — breaks when table columns change

✓ TIPS & BEST PRACTICES

- Always name aggregate columns with `AS` — unnamed aggregates are hard to reference
- Qualify columns with table aliases when joining — avoids ambiguity errors
- Scalar subqueries in `SELECT` run once per row — use sparingly on large tables
- Use `CONCAT_WS()` over `||` when NULLs might be present — it skips them

QUICK SUMMARY

- Defines what columns, expressions, or aggregates to return
- Written first, executed sixth — aliases appear late
- Aggregates require `GROUP BY` alongside non-aggregated columns
- `DISTINCT` deduplicates the full row, not one column

SQL · Sheet 3

WHERE Operators

Comparison · BETWEEN · IN · LIKE · IS NULL · AND / OR / NOT · pattern matching

DESCRIPTION

The `WHERE` clause filters rows before any grouping or aggregation occurs. It accepts one or more conditions combined with logical operators — only rows that evaluate to `TRUE` pass through to the next step. Because `WHERE` executes before `SELECT`, you cannot reference column aliases or aggregate functions here — for aggregate filtering, use `HAVING`. NULL values require special handling: comparisons using `=` or `!=` against NULL always silently return NULL, never TRUE.

NOTES

- `NULL = NULL` is always NULL — never use `=` to check for null
- `AND` is evaluated before `OR` — use parentheses to control order
- `BETWEEN` is inclusive on both ends
- `LIKE` case sensitivity varies by database and collation setting

EXAMPLE

```sql
SELECT * FROM orders
WHERE
    status IN ('pending', 'processing')
    AND total BETWEEN 100 AND 500
    AND customer_name LIKE 'A%'
    AND notes IS NOT NULL
    AND (priority = 'high' OR created_at >= '2024-01-01')
```

CODE EXPLAINED

- → `IN (...)` — matches any value in the list
- → `BETWEEN 100 AND 500` — inclusive range filter
- → `LIKE 'A%'` — starts with A, anything after
- → `IS NOT NULL` — safe null check
- → `(...OR...)` — parentheses force OR to evaluate first before the outer AND

COMPARISON OPERATORS

SYNTAX	DESCRIPTION
`col = value`	Equal to
`col != value`	Not equal (also <>)
`col > value`	Greater than
`col < value`	Less than
`col >= value`	Greater than or equal
`col <= value`	Less than or equal

RANGE, LIST & NULL

SYNTAX	DESCRIPTION
`col BETWEEN a AND b`	Inclusive range
`col NOT BETWEEN a AND b`	Outside range
`col IN (a, b, c)`	Matches any in list
`col NOT IN (a, b, c)`	Matches none in list
`col IN (SELECT ...)`	Matches subquery result
`col IS NULL`	Value is null
`col IS NOT NULL`	Value is not null

LIKE PATTERNS & LOGIC

SYNTAX	DESCRIPTION
`col LIKE 'A%'`	Starts with A
`col LIKE '%A'`	Ends with A
`col LIKE '%A%'`	Contains A
`col LIKE 'A_B'`	A + any one char + B
`cond AND cond`	Both must be true
`cond OR cond`	Either must be true
`NOT cond`	Reverses condition

⚠ COMMON MISTAKES

- Using `col = NULL` — always returns NULL, never TRUE; use `IS NULL`
- Forgetting parentheses with `AND` / `OR` — `AND` binds tighter
- Using aggregate functions in `WHERE` — use `HAVING` instead
- Using a `SELECT` alias in `WHERE` — it hasn't been evaluated yet

✓ TIPS & BEST PRACTICES

- Always use `IS NULL` / `IS NOT NULL` — never `= NULL`
- Use `IN (SELECT ...)` over long `IN (a,b,c...)` lists for maintainability
- Index columns used in `WHERE` for significant performance gains
- Test your `WHERE` with `SELECT COUNT(*)` first to verify row count before fetching data

QUICK SUMMARY

- Filters rows before grouping — runs before `SELECT`
- No aggregates or `SELECT` aliases allowed
- NULL comparisons need `IS NULL` not `= NULL`
- Use parentheses when mixing `AND` and `OR`

SQL · Sheet 4

JOIN Types

INNER · LEFT · RIGHT · FULL · CROSS · SELF JOIN · ON conditions · which rows are kept

DESCRIPTION

A `JOIN` combines rows from two or more tables based on a related column between them. Joins are added after the `FROM` clause and can be chained — each join adds another table to the working dataset. The "left" table is always the one in `FROM`; the "right" table is the one in `JOIN`. The join type determines what happens to rows when no match is found between the two tables.

NOTES

- Always qualify column names with a table alias when joining — avoids ambiguous column errors
- `JOIN` alone is shorthand for `INNER JOIN`
- Chained joins are evaluated left to right
- `CROSS JOIN` on large tables can produce enormous result sets — use with caution

EXAMPLE

```sql
SELECT
    e.first_name, e.last_name,
    d.department_name,
    m.first_name AS manager
FROM       employees AS e
INNER JOIN departments AS d ON e.dept_id = d.id
LEFT JOIN  employees AS m  ON e.manager_id = m.id
WHERE d.location = 'New York'
ORDER BY d.department_name, e.last_name
```

CODE EXPLAINED

- → `FROM employees AS e` — left (main) table
- → `INNER JOIN departments` — only employees with a matching department
- → `LEFT JOIN employees AS m` — include employees even if no manager is assigned
- → `ON e.manager_id = m.id` — self-join: employees table joined to itself
- → `WHERE d.location` — filter applied after all joins

JOIN TYPES

SYNTAX	DESCRIPTION
`INNER JOIN`	Matching rows in both tables only
`LEFT JOIN`	All left rows; unmatched right = NULL
`RIGHT JOIN`	All right rows; unmatched left = NULL
`FULL JOIN`	All rows from both; unmatched = NULL
`CROSS JOIN`	Every combination — no ON needed
`SELF JOIN`	Table joined to itself via aliases

WHICH ROWS ARE RETURNED?

TYPE	LEFT UNMATCHED	MATCHED	RIGHT UNMATCHED
`INNER`	✗	✓	✗
`LEFT`	✓	✓	✗
`RIGHT`	✗	✓	✓
`FULL`	✓	✓	✓
`CROSS`	—	all combos	—

SYNTAX VARIATIONS

SYNTAX	DESCRIPTION
`JOIN`	Shorthand for INNER JOIN
`LEFT OUTER JOIN`	Same as LEFT JOIN
`FULL OUTER JOIN`	Same as FULL JOIN
`JOIN t ON a.id=b.id AND a.type=b.type`	Multiple ON conditions
`LEFT JOIN t ON ... WHERE b.id IS NULL`	Exclude matches (anti-join)

⚠ COMMON MISTAKES

- Not qualifying column names — ambiguous column error when tables share names
- Using `LEFT JOIN` when `INNER JOIN` was intended — silently includes unmatched NULLs
- Forgetting that `WHERE` on a `LEFT JOIN`'s right table converts it to an `INNER JOIN`
- `CROSS JOIN` without a filter — row count explodes multiplicatively

✓ TIPS & BEST PRACTICES

- Always alias every table — even single-table queries benefit from consistency
- Index foreign key columns on both sides of the join for performance
- Use `LEFT JOIN ... WHERE b.id IS NULL` to find rows with no match (anti-join)
- For self-joins, use meaningful aliases: `e` and `m` for employee and manager

QUICK SUMMARY

- `INNER` — only matching rows
- `LEFT` — all left + matched right
- `FULL` — everything from both sides
- Left table = `FROM`, right table = `JOIN`

SQL · Sheet 5

Aggregate Functions

COUNT · SUM · AVG · MIN · MAX · GROUP BY · HAVING · DISTINCT in aggregates

Aggregate functions perform a calculation across a set of rows and return a single value. They are most commonly used in `SELECT` alongside `GROUP BY` to summarize data by category. Without `GROUP BY`, the aggregate treats the entire result set as one group. All aggregate functions ignore `NULL` values except `COUNT(*)`, which counts every row including those with nulls.

NOTES

- Every non-aggregated column in `SELECT` must appear in `GROUP BY`
- `HAVING` filters groups — `WHERE` filters rows; not interchangeable
- `COUNT(*)` is faster than `COUNT(col)` when NULLs don't matter
- Aggregates can be nested inside subqueries for multi-level summarization

EXAMPLE

```sql
SELECT
    department,
    COUNT(*)                    AS total_employees,
    COUNT(DISTINCT job_title)   AS unique_roles,
    AVG(salary)                 AS avg_salary,
    MIN(salary)                 AS lowest_salary,
    MAX(salary)                 AS highest_salary,
    SUM(salary)                 AS payroll_total
FROM     employees
WHERE    status = 'active'
GROUP BY department
HAVING   COUNT(*) > 5
ORDER BY avg_salary DESC
```

CODE EXPLAINED

- → `COUNT(*)` — total rows per group including NULLs
- → `COUNT(DISTINCT)` — unique non-null values only
- → `AVG / MIN / MAX / SUM` — all ignore NULL values
- → `WHERE status` — filters rows before grouping
- → `GROUP BY department` — one row per department
- → `HAVING COUNT(*) > 5` — filters groups after aggregation

CORE FUNCTIONS

FUNCTION	DESCRIPTION
`COUNT(*)`	All rows inc. nulls
`COUNT(col)`	Non-null values only
`COUNT(DISTINCT col)`	Unique non-null values
`SUM(col)`	Total of all values
`AVG(col)`	Average of all values
`MIN(col)`	Smallest value
`MAX(col)`	Largest value

GROUP BY PATTERNS

SYNTAX	DESCRIPTION
`GROUP BY col`	Group by one column
`GROUP BY col1, col2`	Group by multiple columns
`GROUP BY col HAVING AGG() > val`	Filter groups by aggregate
`GROUP BY col HAVING COUNT(*) > 1`	Find duplicates

HAVING PATTERNS

SYNTAX	DESCRIPTION
`HAVING COUNT(*) > 5`	Groups with more than 5 rows
`HAVING SUM(col) > 1000`	Groups where total exceeds value
`HAVING AVG(col) BETWEEN x AND y`	Groups where average is in range
`HAVING MAX(col) != MIN(col)`	Groups where values vary

⚠ COMMON MISTAKES

- Using an aggregate in `WHERE` — use `HAVING` instead
- Non-aggregated columns in `SELECT` missing from `GROUP BY`
- Using `COUNT(col)` when you want to include NULLs — use `COUNT(*)`
- Expecting `AVG` to include NULL rows — it silently skips them

✓ TIPS & BEST PRACTICES

- Use `COUNT(*)` for row counts — it's faster and includes NULLs
- Use `COALESCE(AVG(col), 0)` to handle NULL results from empty groups
- Add `ORDER BY` after `HAVING` to sort summarized results meaningfully
- Nest aggregates in a subquery or CTE for multi-level summaries

QUICK SUMMARY

- Aggregates collapse rows into a single value per group
- All aggregates (except `COUNT(*)`) ignore NULLs
- `GROUP BY` groups · `HAVING` filters groups
- Non-aggregated `SELECT` columns must be in `GROUP BY`

SQL · Sheet 6

CASE Statement

Searched CASE · Simple CASE · WHEN / THEN / ELSE · conditional aggregation · custom sort

DESCRIPTION

The `CASE` statement adds conditional logic to SQL — similar to if/else in programming. It can be used almost anywhere an expression is valid: in `SELECT`, `ORDER BY`, `WHERE`, and inside aggregate functions. There are two forms: the *searched* form evaluates boolean conditions, and the *simple* form compares one column to fixed values. Conditions are evaluated top to bottom — the first match wins.

NOTES

- Conditions are evaluated top to bottom — order matters
- `ELSE` is optional but recommended — omitting it returns `NULL` on no match
- `CASE` is an expression, not a statement — it must return a single value
- Can be nested, but deeply nested `CASE` statements hurt readability

EXAMPLE

```sql
SELECT
    first_name, last_name, salary,
    CASE
        WHEN salary >= 100000 THEN 'Senior'
        WHEN salary >= 60000  THEN 'Mid'
        WHEN salary >= 30000  THEN 'Junior'
        ELSE 'Entry'
    END AS level,
    SUM(CASE WHEN status = 'active' THEN 1 ELSE 0 END) AS active_ct
FROM employees
GROUP BY first_name, last_name, salary
ORDER BY
    CASE level
        WHEN 'Senior' THEN 1
        WHEN 'Mid'    THEN 2
        WHEN 'Junior' THEN 3
        ELSE 4
    END
```

CODE EXPLAINED

- → Searched `CASE` — evaluates salary ranges top to bottom
- → `ELSE 'Entry'` — fallback if no condition matches
- → `AS level` — alias for the CASE result
- → `SUM(CASE...)` — conditional aggregation pattern
- → Simple `CASE` in `ORDER BY` — custom sort order by label

SEARCHED VS SIMPLE

	SEARCHED	SIMPLE
Syntax	CASE WHEN condition	CASE col WHEN value
Operators	Any (>, <, LIKE…)	= only
Best for	Ranges, complex logic	Fixed value lookup

SYNTAX FORMS

FORM	DESCRIPTION
`CASE WHEN cond THEN r END`	Basic — NULL if no match
`CASE WHEN cond THEN r ELSE d END`	With fallback value
`CASE col WHEN val THEN r END`	Simple — compares one column
`CASE WHEN c1 THEN r1 WHEN c2 THEN r2 ELSE r3 END`	Multiple conditions

COMMON USES

USE	PATTERN
Label categories	CASE WHEN col > 0 THEN 'Pos' ELSE 'Zero' END
Conditional aggregate	SUM(CASE WHEN status='active' THEN 1 ELSE 0 END)
Custom sort order	ORDER BY CASE priority WHEN 'high' THEN 1 ELSE 2 END
Pivot rows to columns	SUM(CASE WHEN month='Jan' THEN amount ELSE 0 END)
Avoid divide by zero	CASE WHEN denom=0 THEN NULL ELSE num/denom END

⚠ COMMON MISTAKES

- Omitting `ELSE` — returns NULL silently when no condition matches
- Wrong condition order — putting a broader range before a narrower one
- Using `CASE` in `WHERE` when a regular condition would suffice
- Referencing the `CASE` alias in `HAVING` — it doesn't exist at that step

✓ TIPS & BEST PRACTICES

- Always include `ELSE` to make unmatched behavior explicit
- Order conditions from most specific to most general
- Use conditional aggregation (`SUM(CASE...)`) to pivot data without a subquery
- Use `CASE` in `ORDER BY` for custom sort sequences that can't be expressed numerically

QUICK SUMMARY

- Two forms: searched (conditions) and simple (value match)
- First matching condition wins — order matters
- Usable in `SELECT`, `WHERE`, `ORDER BY`, and inside aggregates
- Always add `ELSE` to avoid silent NULLs

SQL · Sheet 7

String Functions

UPPER · LOWER · TRIM · LENGTH · SUBSTRING · LEFT · RIGHT · CONCAT · REPLACE · CONCAT_WS

DESCRIPTION

String functions manipulate and transform text values in SQL. They can be used in `SELECT` to format output, in `WHERE` to filter on text patterns, or nested inside other functions. Exact function names vary slightly between databases — the most common variations are noted. String indexes in SQL start at **1**, not 0.

NOTES

- String indexes start at `1` in SQL, not 0
- `CONCAT` with a `NULL` returns `NULL` in some databases — use `CONCAT_WS` to skip NULLs
- `||` for concatenation is standard in PostgreSQL / SQLite; use `+` in SQL Server
- `LENGTH` in MySQL / PostgreSQL; `LEN` in SQL Server

EXAMPLE

```sql
SELECT
    CONCAT_WS(' ', first_name, last_name)  AS full_name,
    UPPER(department)                       AS dept,
    LENGTH(TRIM(email))                     AS email_length,
    LEFT(phone, 3)                          AS area_code,
    REPLACE(notes, 'N/A', 'None')           AS cleaned_notes,
    SUBSTRING(employee_id, 1, 4)            AS id_prefix
FROM  employees
WHERE TRIM(email) != ''
ORDER BY full_name
```

CODE EXPLAINED

- → `CONCAT_WS(' ', ...)` — joins with space separator, skips NULLs
- → `UPPER()` — converts dept to uppercase
- → `LENGTH(TRIM())` — nested: strip spaces then measure
- → `LEFT(phone, 3)` — first 3 characters only
- → `REPLACE()` — swap all occurrences of a substring
- → `SUBSTRING(col, 1, 4)` — chars 1 through 4

CASE & WHITESPACE

FUNCTION	DESCRIPTION
`UPPER(col)`	Convert to uppercase
`LOWER(col)`	Convert to lowercase
`TRIM(col)`	Remove leading & trailing spaces
`LTRIM(col)`	Remove leading spaces only
`RTRIM(col)`	Remove trailing spaces only

EXTRACTING & REPLACING

FUNCTION	DESCRIPTION
`SUBSTRING(col, s, len)`	Extract part of a string
`LEFT(col, n)`	First n characters
`RIGHT(col, n)`	Last n characters
`REPLACE(col, 'old', 'new')`	Replace all occurrences
`REVERSE(col)`	Reverse the string

LENGTH, POSITION & COMBINE

FUNCTION	DESCRIPTION
`LENGTH(col)`	Character count (LEN in SQL Server)
`POSITION('x' IN col)`	Position of substring
`CONCAT(col1, col2)`	Join two strings
`col1 || col2`	Concatenation — PostgreSQL / SQLite
`CONCAT_WS(sep, c1, c2)`	Join with separator, skips NULLs

⚠ COMMON MISTAKES

- Using index 0 in `SUBSTRING` — SQL strings start at 1
- Using `CONCAT(col, NULL)` — returns NULL in MySQL; use `CONCAT_WS`
- Forgetting `TRIM` before comparing strings — trailing spaces cause mismatches
- Using `LENGTH` in SQL Server — it's `LEN` there

✓ TIPS & BEST PRACTICES

- Use `CONCAT_WS` instead of `CONCAT` when any value might be NULL
- Nest functions freely: `UPPER(TRIM(col))` is clean and readable
- Use `TRIM` in `WHERE` clauses when comparing user-entered data
- Prefer `||` in PostgreSQL for readability over nested `CONCAT` calls

QUICK SUMMARY

- String indexes start at 1, not 0
- `CONCAT_WS` is safer than `CONCAT` when NULLs exist
- Function names vary by database — check `LEN` vs `LENGTH`
- Functions can be nested for multi-step transformations

Date Functions

NOW · CURDATE · YEAR · MONTH · DAY · EXTRACT · DATE_ADD · DATEDIFF · formatting

DESCRIPTION

Date functions retrieve, calculate, and format date and time values. They are among the most database-specific functions in SQL — the same concept often has a different name or syntax in MySQL, PostgreSQL, and SQL Server. The most common variations are noted throughout. Date literals should always be written as quoted strings in ISO format: `'2024-01-01'`. Always store dates as a `DATE` or `TIMESTAMP` type — never as strings.

NOTES

- `DATEDIFF` argument order varies by database — check which is start vs end
- Time zones can affect `NOW()` — be explicit when precision matters
- `EXTRACT` is the ANSI standard — works across most databases
- Storing dates as strings disables all date functions and comparisons

EXAMPLE

```sql
SELECT
    first_name,
    hire_date,
    YEAR(hire_date)                          AS hire_year,
    EXTRACT(MONTH FROM hire_date)            AS hire_month,
    DATEDIFF(NOW(), hire_date)               AS days_employed,
    DATE_ADD(hire_date, INTERVAL 90 DAY)   AS end_of_probation
FROM  employees
WHERE hire_date >= '2023-01-01'
  AND hire_date <= NOW()
ORDER BY hire_date DESC
```

CODE EXPLAINED

→ `YEAR(hire_date)` — extract year as integer

→ `EXTRACT(MONTH FROM...)` — ANSI standard extraction

→ `DATEDIFF(NOW(), hire_date)` — days between two dates

→ `DATE_ADD(..., INTERVAL 90 DAY)` — add 90 days to a date

→ `hire_date >= '2023-01-01'` — ISO date literal comparison

CURRENT DATE & TIME

FUNCTION	DESCRIPTION
`NOW()`	Date + time (GETDATE() in SQL Server)
`CURDATE()`	Date only (CURRENT_DATE in PostgreSQL)
`CURTIME()`	Time only (CURRENT_TIME in PostgreSQL)
`SYSDATE()`	Execution-time datetime — MySQL / Oracle

EXTRACTING & CALCULATING

FUNCTION	DESCRIPTION
`YEAR(col)`	Extract the year
`MONTH(col)`	Extract month as number
`DAY(col)`	Extract day of month
`EXTRACT(YEAR FROM col)`	ANSI standard extraction
`DATEDIFF(col1, col2)`	Days between two dates — MySQL
`col1 - col2`	Difference as interval — PostgreSQL

ADDING, SUBTRACTING & FORMATTING

FUNCTION	DESCRIPTION
`DATE_ADD(col, INTERVAL n DAY)`	Add n days — MySQL
`DATE_SUB(col, INTERVAL n MONTH)`	Subtract months — MySQL
`col + INTERVAL '1 year'`	Add one year — PostgreSQL
`DATEADD(day, n, col)`	Add days — SQL Server
`DATE_FORMAT(col, '%Y-%m-%d')`	Format date — MySQL
`TO_CHAR(col, 'YYYY-MM-DD')`	Format date — PostgreSQL

⚠ COMMON MISTAKES

- Storing dates as strings — disables all date arithmetic and comparisons
- Wrong argument order in `DATEDIFF` — result may be negative unexpectedly
- Ignoring time zones with `NOW()` — results vary by server timezone
- Using `YEAR(col) = 2024` instead of a range — prevents index usage

✓ TIPS & BEST PRACTICES

- Use `EXTRACT` for cross-database compatibility
- Use range comparisons (`BETWEEN`) instead of `YEAR(col) =` to allow index use
- Always use ISO format for date literals: `'2024-01-01'`
- Store timestamps as `TIMESTAMP` , not `DATETIME` , for timezone awareness

QUICK SUMMARY

- Date functions are the most database-specific area of SQL
- `EXTRACT` is the safest cross-database option
- Always store dates as `DATE` / `TIMESTAMP` , never strings
- Use ISO format `'YYYY-MM-DD'` for all date literals

SQL · Sheet 9

UNION / INTERSECT / EXCEPT

UNION · UNION ALL · INTERSECT · EXCEPT · combining result sets · duplicate handling

DESCRIPTION

Set operators combine the results of two or more `SELECT` statements into a single result set — stacking rows vertically, unlike `JOIN` which combines columns horizontally. Both queries must return the same number of columns, in the same order, with compatible data types. Column names in the final result are taken from the first query. `ORDER BY` and `LIMIT` go at the very end and apply to the combined result.

NOTES

- Column count and data types must match across all queries
- Column names come from the first `SELECT` — alias there for custom names
- `UNION ALL` is significantly faster than `UNION` — only deduplicate when necessary
- `INTERSECT` and `EXCEPT` not supported in MySQL before version 8.0

EXAMPLE

```sql
-- All customers and suppliers in one list
SELECT name, city, 'Customer' AS type FROM customers
UNION ALL
SELECT name, city, 'Supplier' AS type FROM suppliers
ORDER BY city, name
-- Customers who are also suppliers
SELECT email FROM customers
INTERSECT
SELECT email FROM suppliers
-- Customers who are NOT suppliers
SELECT email FROM customers
EXCEPT
SELECT email FROM suppliers
```

CODE EXPLAINED

→ `UNION ALL` — stacks both results, keeps duplicates

→ `'Customer' AS type` — static label to identify source

→ `ORDER BY` — applied to the entire combined result

→ `INTERSECT` — only emails appearing in both tables

→ `EXCEPT` — emails in customers but not in suppliers

SET OPERATORS

OPERATOR	DESCRIPTION
`UNION`	Combine results, remove duplicates
`UNION ALL`	Combine results, keep all rows
`INTERSECT`	Rows appearing in both results
`EXCEPT`	Rows in first but not second (MINUS in Oracle)

⚠ COMMON MISTAKES

- Mismatched column count between queries — throws an error
- Using `UNION` when `UNION ALL` would do — unnecessary deduplication cost
- Putting `ORDER BY` on individual queries instead of the final result
- Using `INTERSECT` / `EXCEPT` on MySQL < 8.0 — not supported

BEHAVIOR COMPARISON

	UNION	UNION ALL	INTERSECT	EXCEPT
Removes dupes	✓	✗	✓	✓
Keeps all rows	✗	✓	✗	✗
Order matters	✗	✗	✗	✓
Performance	Higher	Lower	Higher	Higher

✓ TIPS & BEST PRACTICES

- Default to `UNION ALL` — use `UNION` only when deduplication is needed
- Add a literal column to identify which query each row came from
- Alias columns in the first query for readable output headers
- Wrap individual queries in parentheses for readability in complex unions

RULES & PATTERNS

RULE	DESCRIPTION
`Column count`	Must match across all queries
`Data types`	Must be compatible per column
`Column names`	Taken from first SELECT
`ORDER BY`	One at the end — applies to all
`Parentheses`	Wrap queries for clarity

QUICK SUMMARY

- Stacks rows vertically — unlike JOIN which adds columns
- All queries must have same column count and compatible types
- `UNION ALL` is faster — use `UNION` only to deduplicate
- One `ORDER BY` at the end applies to the whole result

SQL · Sheet 10

Subqueries

Scalar · IN · EXISTS · derived tables · correlated vs non-correlated · anti-join

DESCRIPTION

A subquery is a `SELECT` statement nested inside another query. It is evaluated first and its result is passed to the outer query. Subqueries can appear in `SELECT`, `FROM`, or `WHERE` depending on what they return — a single value, a list of values, or a full table. A *correlated* subquery references the outer query and runs once per row; a *non-correlated* subquery is independent and runs once.

NOTES

- Scalar subqueries that return more than one row will throw an error
- `EXISTS` is generally faster than `IN` for large datasets
- Correlated subqueries can be slow — consider rewriting as a `JOIN` or CTE
- Derived tables in `FROM` must always have an alias

EXAMPLE — CORRELATED SUBQUERY

```sql
-- Employees earning above their department average
SELECT first_name, last_name, salary, department
FROM   employees e
WHERE  salary > (
    SELECT AVG(salary)
    FROM   employees
    WHERE  department = e.department
)
ORDER BY department, salary DESC
```

CODE EXPLAINED

- → `e` — alias for the outer query's table
- → Inner `SELECT AVG(salary)` — runs once per outer row
- → `WHERE department = e.department` — references the outer query (correlated)
- → Result — only employees above their own dept average

CORRELATED VS NON-CORRELATED

	NON-CORRELATED	CORRELATED
Runs	Once, independently	Once per outer row
References outer	✗	✓
Performance	Fast	Can be slow
Rewrite as	Stays as subquery	JOIN or CTE

COMMON PATTERNS

PATTERN	DESCRIPTION
`WHERE col = (SELECT MAX(col) FROM t)`	Filter by aggregate
`WHERE NOT EXISTS (SELECT 1 FROM t WHERE...)`	Anti-join / no match
`WHERE col > (SELECT AVG(col) FROM t)`	Compare to overall avg
`FROM (SELECT...) AS sub WHERE sub.col > val`	Filter on aggregated result

SUBQUERY BY LOCATION

LOCATION	RETURNS	DESCRIPTION
`WHERE col = (SELECT...)`	Single value	Scalar — one row, one column
`WHERE col IN (SELECT...)`	List	Multiple rows, one column
`WHERE EXISTS (SELECT...)`	True/False	True if any rows returned
`FROM (SELECT...) AS alias`	Table	Derived table — must be aliased
`SELECT (SELECT...)`	Single value	Scalar inline in SELECT

⚠ COMMON MISTAKES

- Scalar subquery returning more than one row — causes a runtime error
- Forgetting to alias derived tables in `FROM`
- Using a correlated subquery where a `JOIN` would be much faster
- Using `IN (SELECT...)` with NULLs in the subquery — `NOT IN` returns no rows

✓ TIPS & BEST PRACTICES

- Prefer `EXISTS` over `IN` for large subquery result sets
- Rewrite slow correlated subqueries as CTEs or JOINs
- Use `NOT EXISTS` instead of `NOT IN` when NULLs may be present
- Consider a CTE when the same subquery is referenced more than once

QUICK SUMMARY

- Nested SELECT — evaluated before the outer query
- Location determines return type: value, list, or table
- Correlated = references outer query, runs per row
- Derived tables in `FROM` must always be aliased

SQL · Sheet 11

CTEs (Common Table Expressions)

WITH clause · multiple CTEs · recursive CTEs · anchor member · CTE vs subquery

DESCRIPTION

A CTE is a temporary named result set defined at the top of a query using the `WITH` keyword. It exists only for the duration of the query and can be referenced like a table in the `SELECT`, `FROM`, `WHERE`, or `JOIN` clauses that follow. CTEs improve readability by breaking complex queries into named, logical steps — and unlike subqueries, a CTE can reference itself, enabling recursive traversal of hierarchical data.

NOTES

- CTEs are not stored — they exist only for the current query
- Multiple CTEs use commas, not repeated `WITH` keywords
- Recursive CTEs require a termination condition to avoid infinite loops
- Some databases allow `INSERT`, `UPDATE`, or `DELETE` after a CTE

EXAMPLE — MULTIPLE CTES

```sql
WITH dept_averages AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM   employees
    GROUP BY department
),
high_earners AS (
    SELECT e.first_name, e.last_name, e.salary, e.department
    FROM   employees e
    JOIN   dept_averages d ON e.department = d.department
    WHERE  e.salary > d.avg_salary
)
SELECT *
FROM   high_earners
ORDER BY department, salary DESC
```

CODE EXPLAINED

- → `WITH dept_averages AS (...)` — first CTE: avg salary per dept
- → Comma separates CTEs — no second `WITH`
- → `high_earners AS (...)` — second CTE references the first
- → `JOIN dept_averages` — CTEs used like regular tables
- → Final `SELECT` — queries the second CTE as if it's a table

SYNTAX

SYNTAX	DESCRIPTION
`WITH cte AS (SELECT...)`	Define a single CTE
`... SELECT * FROM cte`	Use CTE like a table
`WITH c1 AS (...), c2 AS (...)`	Multiple CTEs — comma separated
`WITH RECURSIVE cte AS (...)`	Recursive — can reference itself

CTE VS SUBQUERY

	CTE	SUBQUERY
Readability	✓ Named, clear	✗ Can nest deeply
Reusable	✓	✗
Recursive	✓	✗
Performance	Similar	Similar

RECURSIVE CTE STRUCTURE

PART	DESCRIPTION
`Anchor member`	Starting point — non-recursive SELECT
`UNION ALL`	Connects anchor to recursive part
`Recursive member`	References the CTE itself
`Termination`	WHERE clause that stops recursion

⚠ COMMON MISTAKES

- Using multiple `WITH` keywords — only one `WITH`, separate CTEs with commas
- Recursive CTE without a termination condition — infinite loop
- Expecting CTEs to be cached/persisted — they re-evaluate each time referenced
- Referencing a CTE before it is defined — CTEs are evaluated in order

✓ TIPS & BEST PRACTICES

- Use CTEs to name intermediate steps — makes complex queries self-documenting
- Prefer CTEs over subqueries when the same result is needed more than once
- Keep each CTE focused on one logical step
- Add a `LIMIT` or `MAX_DEPTH` guard to recursive CTEs during development

QUICK SUMMARY

- Temporary named result set for the duration of one query
- Multiple CTEs: one `WITH`, comma-separated
- More readable and reusable than nested subqueries
- Recursive form enables hierarchical data traversal

SQL · Sheet 12

Window Functions

ROW_NUMBER · RANK · DENSE_RANK · OVER · PARTITION BY · LAG · LEAD · window frames

DESCRIPTION

Window functions perform calculations across rows related to the current row — like aggregates, but without collapsing rows into a single result. Each row keeps its identity while gaining access to values from other rows in the defined "window." They are always used in `SELECT` and require an `OVER()` clause to define the window. They cannot be used in `WHERE` or `HAVING` — wrap in a CTE or subquery to filter on their results.

NOTES

- `PARTITION BY` is optional — omit to treat entire result as one window
- `ORDER BY` inside `OVER()` is independent of the query's `ORDER BY`
- Cannot filter on window function results in `WHERE` — use a CTE
- `RANK()` skips numbers after ties; `DENSE_RANK()` does not

EXAMPLE

```
SELECT
    department, first_name, salary,
    RANK()        OVER (PARTITION BY department ORDER BY salary DESC)
                  AS dept_rank,
    DENSE_RANK()  OVER (PARTITION BY department ORDER BY salary DESC)
                  AS dense_rank,
    AVG(salary)   OVER (PARTITION BY department)
                  AS dept_avg,
    SUM(salary)   OVER (PARTITION BY department ORDER BY salary)
                  AS running_total,
    LAG(salary,1) OVER (PARTITION BY department ORDER BY salary)
                  AS prev_salary
FROM employees
ORDER BY department, salary DESC
```

CODE EXPLAINED

- → `PARTITION BY department` — resets window per department
- → `RANK()` — rank with gaps after ties
- → `DENSE_RANK()` — rank without gaps after ties
- → `AVG() OVER (PARTITION BY...)` — dept avg on every row
- → `SUM() OVER (...ORDER BY)` — running total within dept
- → `LAG(salary,1)` — previous row's salary value

RANKING FUNCTIONS

FUNCTION	DESCRIPTION
`ROW_NUMBER()`	Unique sequential number — no ties
`RANK()`	Rank with gaps after ties
`DENSE_RANK()`	Rank without gaps after ties
`NTILE(n)`	Divide rows into n equal buckets
`PERCENT_RANK()`	Relative rank as 0–1 percentage

OVER() CLAUSES & FRAMES

SYNTAX	DESCRIPTION
`OVER ()`	Entire result set as one window
`OVER (PARTITION BY col)`	Separate window per group
`OVER (ORDER BY col)`	Ordered window, running calc
`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`	From start to current row
`ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING`	Current ± 1 row

AGGREGATE & VALUE FUNCTIONS

FUNCTION	DESCRIPTION
`SUM(col) OVER (...)`	Running total within window
`AVG(col) OVER (...)`	Average without collapsing rows
`COUNT(*) OVER (...)`	Count without collapsing rows
`LAG(col, n) OVER (...)`	Value n rows before current
`LEAD(col, n) OVER (...)`	Value n rows after current
`FIRST_VALUE(col) OVER (...)`	First value in the window
`LAST_VALUE(col) OVER (...)`	Last value in the window

⚠ COMMON MISTAKES

- Using a window function in `WHERE` — wrap in a CTE first
- Confusing `RANK()` and `DENSE_RANK()` — gaps vs no gaps after ties
- Omitting `ORDER BY` inside `OVER()` for `LAG` / `LEAD` — result is undefined
- Assuming `OVER()` `ORDER BY` sorts the final output — it doesn't

✓ TIPS & BEST PRACTICES

- Use `ROW_NUMBER()` to deduplicate — pick the first row per group
- Use `LAG` / `LEAD` for period-over-period comparisons without a self-join
- Wrap window function results in a CTE when you need to filter them
- Reuse the same window with `WINDOW w AS (...)` clause (PostgreSQL)

QUICK SUMMARY

- Like aggregates, but rows are not collapsed
- Always requires `OVER()` clause
- Cannot be used in `WHERE` or `HAVING`
- `PARTITION BY` resets the window per group

SQL · Sheet 13

NULL Handling

IS NULL · IS NOT NULL · COALESCE · NULLIF · ISNULL · IFNULL · NULL in aggregates & logic

DESCRIPTION

`NULL` represents the absence of a value — it is not zero, not an empty string, and not false. Comparisons with `NULL` using `=` or `!=` always return `NULL` — never TRUE or FALSE — so standard operators silently fail. Most databases provide functions specifically designed to detect and substitute NULL values. `COALESCE` is the ANSI standard and works across all major databases.

NOTES

- `NULL = NULL` returns NULL — always use `IS NULL` to check
- Joining on a NULL column will never match — NULLs don't equal each other
- `NULL` values sort last in `ASC` and first in `DESC` in most databases
- Prefer `COALESCE` over `ISNULL` / `IFNULL` — it's ANSI standard

EXAMPLE

```
SELECT
    first_name,
    COALESCE(phone, email, 'No contact')   AS contact,
    COALESCE(manager_id, 0)                AS manager,
    NULLIF(bonus, 0)                       AS bonus,
    salary / NULLIF(hours_worked, 0)       AS hourly_rate
FROM  employees
WHERE phone IS NULL OR email IS NOT NULL
ORDER BY last_name
```

CODE EXPLAINED

- → `COALESCE(phone, email, 'No contact')` — first non-null wins
- → `COALESCE(manager_id, 0)` — replace NULL with 0
- → `NULLIF(bonus, 0)` — treat 0 as NULL (no bonus)
- → `salary / NULLIF(hours_worked, 0)` — safe division, avoids divide-by-zero error
- → `IS NULL / IS NOT NULL` — correct NULL checks

CHECKING FOR NULL

SYNTAX	DESCRIPTION
`col IS NULL`	True if value is null
`col IS NOT NULL`	True if value is not null
`col = NULL`	Always returns NULL — never use
`col != NULL`	Always returns NULL — never use

NULL HANDLING FUNCTIONS

FUNCTION	DESCRIPTION
`COALESCE(c1, c2, c3)`	First non-null — ANSI standard
`NULLIF(col, value)`	Return NULL if col equals value
`ISNULL(col, replacement)`	Replace null — SQL Server
`IFNULL(col, replacement)`	Replace null — MySQL
`NVL(col, replacement)`	Replace null — Oracle

NULL IN AGGREGATES & LOGIC

BEHAVIOR	RESULT
`SUM / AVG / MIN / MAX`	Ignore NULLs
`COUNT(col)`	Ignores NULLs
`COUNT(*)`	Counts all rows inc. NULLs
`NULL AND TRUE`	Returns NULL
`NULL OR TRUE`	Returns TRUE
`NULL = NULL`	Returns NULL

⚠ COMMON MISTAKES

- Using `= NULL` or `!= NULL` — always use `IS NULL` / `IS NOT NULL`
- Using `NOT IN (subquery)` when subquery may contain NULLs — returns no rows
- Joining on nullable columns — NULLs never match, rows silently disappear
- Expecting `AVG` to count NULL rows — it skips them, skewing the average

✓ TIPS & BEST PRACTICES

- Use `COALESCE` over database-specific functions for portability
- Use `NULLIF(denominator, 0)` to safely prevent division by zero
- Use `NOT EXISTS` instead of `NOT IN` when subquery results may contain NULLs
- Add `NOT NULL` constraints at the schema level to prevent NULLs where not needed

QUICK SUMMARY

- NULL = absence of value — not zero, not empty string
- Always check with `IS NULL`, never `= NULL`
- `COALESCE` is the ANSI-standard NULL replacement
- Most aggregates silently ignore NULLs

INSERT

Single row · multiple rows · INSERT SELECT · DEFAULT · NULL · upsert · ON CONFLICT

DESCRIPTION

The `INSERT` statement adds new rows to a table. You can insert a single row with explicit values, multiple rows in one statement, or the results of a `SELECT` query directly into a table. Always list column names explicitly — relying on column order will break if the table structure ever changes. Auto-increment and serial columns should be omitted from the column list.

NOTES

- Always list column names explicitly — never rely on column order
- `INSERT SELECT` does not use `VALUES` — the `SELECT` replaces it
- Wrapping multiple inserts in a transaction is significantly faster
- Use `RETURNING col` in PostgreSQL to get inserted values back

EXAMPLE

```sql
-- Single row
INSERT INTO employees (first_name, last_name, department, salary)
VALUES ('Jane', 'Smith', 'Engineering', 85000)
-- Multiple rows
INSERT INTO employees (first_name, last_name, department, salary)
VALUES
    ('John', 'Doe',   'Marketing',   65000),
    ('Sara', 'Jones', 'Engineering', 92000)
-- From a query
INSERT INTO employees_archive (first_name, last_name, department)
SELECT first_name, last_name, department
FROM   employees
WHERE  status = 'inactive'
```

CODE EXPLAINED

- → Column list explicitly named — safe against schema changes
- → Single `VALUES` — one row inserted
- → Multi-row — one `VALUES` with comma-separated tuples
- → `INSERT SELECT` — no `VALUES` keyword used
- → `WHERE status = 'inactive'` — filtered source rows

PERFORMANCE PATTERNS

PATTERN	DESCRIPTION
`Batch inserts`	Insert multiple rows per statement
`Transaction wrap`	BEGIN / COMMIT around bulk inserts
`INSERT SELECT`	Faster than row-by-row for bulk copy
`RETURNING *`	Get inserted rows back — PostgreSQL

SYNTAX FORMS

SYNTAX	DESCRIPTION
`INSERT INTO t (col) VALUES (val)`	Single row — named columns
`INSERT INTO t VALUES (v1, v2, v3)`	Single row — all columns in order
`INSERT INTO t (col) VALUES (v1), (v2)`	Multiple rows
`INSERT INTO t (col) SELECT col FROM other`	From a query — no VALUES
`INSERT INTO t (col) VALUES (DEFAULT)`	Use column default value
`INSERT INTO t (col) VALUES (NULL)`	Explicitly insert null

UPSERT / ON CONFLICT

SYNTAX	DESCRIPTION
`INSERT OR IGNORE INTO t ...`	Skip on conflict — SQLite
`INSERT OR REPLACE INTO t ...`	Replace on conflict — SQLite
`... ON DUPLICATE KEY UPDATE col=val`	Update on duplicate — MySQL
`... ON CONFLICT (col) DO NOTHING`	Skip on conflict — PostgreSQL
`... ON CONFLICT (col) DO UPDATE SET col=val`	Upsert — PostgreSQL

⚠ COMMON MISTAKES

- Relying on column order instead of naming columns — breaks on schema changes
- Including auto-increment columns in the insert — use DEFAULT or omit
- Using `INSERT SELECT` with `VALUES` — the keyword is not used with SELECT
- Not wrapping bulk inserts in a transaction — dramatically slower row-by-row

✓ TIPS & BEST PRACTICES

- Always name columns explicitly in every INSERT
- Use multi-row `VALUES` or `INSERT SELECT` for bulk operations
- Wrap large batch inserts in `BEGIN` / `COMMIT` for speed
- Use upsert patterns when duplicate key conflicts are expected

QUICK SUMMARY

- Always name columns — never rely on order
- Multi-row inserts are faster than individual inserts
- `INSERT SELECT` has no `VALUES` keyword
- Upsert syntax varies significantly by database

SQL · Sheet 15

UPDATE and DELETE

UPDATE SET · UPDATE from another table · DELETE · TRUNCATE · safety patterns

DESCRIPTION

UPDATE modifies existing rows and DELETE removes them permanently. Both require a WHERE clause to target specific rows — running either without one will affect every row in the table. Always test your WHERE condition with a SELECT first to verify which rows will be affected before committing changes.

NOTES

- `UPDATE` / `DELETE` without `WHERE` affects every row — no undo outside a transaction
- `TRUNCATE` is faster than `DELETE` for clearing all rows but cannot be filtered
- Always run a `SELECT` with your `WHERE` first to verify the target rows
- Use `RETURNING *` in PostgreSQL to see which rows were affected

EXAMPLE

```sql
-- Update a single row
UPDATE employees
SET    salary = 90000, updated_at = NOW()
WHERE  id = 42

-- Update multiple rows
UPDATE employees
SET    status = 'inactive'
WHERE  last_login < '2023-01-01'
  AND  status = 'active'

-- Safe delete: verify first
SELECT * FROM employees
WHERE  status = 'inactive' AND hire_date < '2020-01-01'
-- If correct, run the DELETE:
DELETE FROM employees
WHERE  status = 'inactive' AND hire_date < '2020-01-01'
```

CODE EXPLAINED

- → `SET salary = 90000` — set one column value
- → `SET col1 = v1, col2 = v2` — set multiple columns in one UPDATE
- → `WHERE id = 42` — target by primary key for precision
- → `SELECT *` with same `WHERE` — verify rows before deleting
- → `DELETE FROM` — identical `WHERE` as the verification SELECT

⚠ SAFETY WORKFLOW

STEP	ACTION
1	Run SELECT with your WHERE to see affected rows
2	Wrap in BEGIN / ROLLBACK to test
3	Replace ROLLBACK with COMMIT when satisfied
4	Target by primary key for single-row precision

UPDATE SYNTAX

SYNTAX	DESCRIPTION
`UPDATE t SET col=val WHERE cond`	Update a single column
`UPDATE t SET c1=v1, c2=v2 WHERE cond`	Update multiple columns
`UPDATE t SET col=col+1 WHERE cond`	Update using existing value
`UPDATE t SET col=NULL WHERE cond`	Set a column to null
`UPDATE t SET col=(SELECT...) WHERE cond`	Set from a subquery
`UPDATE t1 SET t1.col=t2.col FROM t1 JOIN t2 ON...`	Update from another table

DELETE & TRUNCATE

SYNTAX	DESCRIPTION
`DELETE FROM t WHERE cond`	Delete specific rows
`DELETE FROM t`	Delete ALL rows — no undo
`TRUNCATE TABLE t`	Delete all rows — faster, may not rollback
`DELETE FROM t1 USING t2 WHERE...`	Delete using join — PostgreSQL
`DELETE t1 FROM t1 JOIN t2 ON...`	Delete using join — MySQL

⚠ COMMON MISTAKES

- Running `UPDATE` or `DELETE` without a `WHERE` — affects every row
- Not testing the `WHERE` with a `SELECT` first
- Using `TRUNCATE` when you need the ability to roll back
- Updating based on a JOIN without testing — may affect far more rows than expected

✓ TIPS & BEST PRACTICES

- Always verify affected rows with `SELECT COUNT(*)` before executing
- Test in a transaction: `BEGIN; UPDATE...; SELECT...; ROLLBACK;`
- Use `WHERE id = ?` (primary key) for single-row precision
- Consider soft deletes (`status = 'deleted'`) for audit trails

QUICK SUMMARY

- Always use `WHERE` — no WHERE = all rows affected
- Test `WHERE` with `SELECT` before modifying data
- `TRUNCATE` is fastest but cannot be filtered or rolled back
- Wrap risky operations in a transaction for safety

SQL · Sheet 16

CREATE TABLE & Data Types

CREATE TABLE · ALTER TABLE · DROP TABLE · numeric · string · date · other types

DESCRIPTION

The `CREATE TABLE` statement defines a new table — its columns, data types, and constraints. Choosing the right data type for each column matters for storage efficiency, query performance, and data integrity. Always define a primary key — tables without one are harder to maintain, join, and query. Use `IF NOT EXISTS` to avoid errors when running setup scripts multiple times.

NOTES

- Use `DECIMAL` not `FLOAT` for money — floating point is imprecise
- `TIMESTAMP` is preferred over `DATETIME` for timezone awareness
- `VARCHAR(255)` is a common default — set n to what the data actually needs
- `DROP TABLE` is permanent — always use `IF EXISTS` in scripts

EXAMPLE

```
CREATE TABLE IF NOT EXISTS employees (
    id          INT             PRIMARY KEY AUTO_INCREMENT,
    first_name  VARCHAR(50)     NOT NULL,
    last_name   VARCHAR(50)     NOT NULL,
    email       VARCHAR(100)    NOT NULL UNIQUE,
    department  VARCHAR(50),
    salary      DECIMAL(10, 2)  DEFAULT 0.00,
    hire_date   DATE            NOT NULL,
    is_active   BOOLEAN         DEFAULT TRUE,
    created_at  TIMESTAMP       DEFAULT CURRENT_TIMESTAMP
)
```

CODE EXPLAINED

- → `IF NOT EXISTS` — safe to run in setup scripts
- → `AUTO_INCREMENT` — auto-generates IDs (SERIAL in PostgreSQL)
- → `NOT NULL` — column must always have a value
- → `UNIQUE` — enforces no duplicate emails
- → `DECIMAL(10, 2)` — exact money values, 2 decimal places
- → `DEFAULT CURRENT_TIMESTAMP` — auto-set on insert

TABLE MANAGEMENT

SYNTAX	DESCRIPTION
`CREATE TABLE t (...)`	Create a new table
`CREATE TABLE IF NOT EXISTS t`	Only create if not exists
`CREATE TABLE new AS SELECT * FROM old`	Create from query result
`CREATE TEMPORARY TABLE t`	Exists for current session only
`DROP TABLE t`	Permanently delete table & data
`DROP TABLE IF EXISTS t`	Delete only if it exists
`ALTER TABLE t ADD COLUMN col type`	Add a column
`ALTER TABLE t DROP COLUMN col`	Remove a column
`ALTER TABLE t RENAME TO new_name`	Rename a table

NUMERIC TYPES

TYPE	DESCRIPTION
`INT / INTEGER`	Whole numbers — 4 bytes
`BIGINT`	Large whole numbers — 8 bytes
`SMALLINT`	Small whole numbers — 2 bytes
`DECIMAL(p, s)`	Exact decimal — use for money
`FLOAT / REAL`	Approximate — avoid for money

STRING, DATE & OTHER TYPES

`CHAR(n)`	Fixed length — padded with spaces
`VARCHAR(n)`	Variable length — up to n chars
`TEXT`	Unlimited length string
`DATE`	Date only — YYYY-MM-DD
`TIMESTAMP`	Date + time, timezone aware
`BOOLEAN`	True / false (1/0 in some DBs)
`UUID`	Universally unique identifier
`JSON`	JSON data — PostgreSQL / MySQL 5.7+

⚠ COMMON MISTAKES

- Using `FLOAT` for money — floating point precision errors cause incorrect totals
- No primary key — harder to update, delete, join, and index
- `DROP TABLE` without `IF EXISTS` — errors in scripts if table doesn't exist
- Using `TEXT` everywhere instead of appropriately sized `VARCHAR`

✓ TIPS & BEST PRACTICES

- Always define a primary key — every table needs one
- Use `DECIMAL(p,s)` for any monetary or precise numeric values
- Use `TIMESTAMP DEFAULT CURRENT_TIMESTAMP` for audit columns
- Add `IF NOT EXISTS` / `IF EXISTS` to all DDL for idempotent scripts

QUICK SUMMARY

- Always define a primary key
- `DECIMAL` not `FLOAT` for money
- Use `IF NOT EXISTS` for safe scripting
- `DROP TABLE` is permanent — use with care

SQL · Sheet 17

Constraints

PRIMARY KEY · FOREIGN KEY · NOT NULL · UNIQUE · DEFAULT · CHECK · ON DELETE CASCADE

DESCRIPTION

Constraints enforce rules on the data in a table, ensuring accuracy and integrity at the database level. They can be defined inline with a column or at the table level after all columns are listed. When a constraint is violated, the database rejects the operation with an error. Naming your constraints explicitly makes them much easier to identify and drop later with `ALTER TABLE ... DROP CONSTRAINT name`.

NOTES

- A table can have only one `PRIMARY KEY` but it can span multiple columns
- `ON DELETE CASCADE` silently deletes all child rows — use with care
- Name constraints explicitly — unnamed constraints are hard to drop later
- `CHECK` constraints are ignored in some older MySQL versions

EXAMPLE

```
CREATE TABLE IF NOT EXISTS orders (
    id          INT             PRIMARY KEY AUTO_INCREMENT,
    customer_id INT             NOT NULL,
    total       DECIMAL(10, 2)  NOT NULL CHECK (total >= 0),
    status      VARCHAR(20)     NOT NULL DEFAULT 'pending',
    created_at  TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_customer FOREIGN KEY (customer_id)
        REFERENCES customers(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    CONSTRAINT chk_status CHECK
        (status IN ('pending', 'processing', 'complete', 'cancelled'))
)
```

CODE EXPLAINED

- → `PRIMARY KEY AUTO_INCREMENT` — unique, auto-generated ID
- → `NOT NULL` — column must always have a value
- → `CHECK (total >= 0)` — inline check constraint
- → `DEFAULT 'pending'` — fallback when not provided
- → `CONSTRAINT fk_customer` — named foreign key for easy management
- → `ON DELETE CASCADE` — deletes orders if customer deleted

PRIMARY KEY VARIATIONS

SYNTAX	DESCRIPTION
`id INT PRIMARY KEY`	Single column — inline
`PRIMARY KEY (col1, col2)`	Composite — table level
`id INT AUTO_INCREMENT PK`	Auto-incrementing — MySQL
`id SERIAL PRIMARY KEY`	Auto-incrementing — PostgreSQL
`id INT IDENTITY(1,1) PK`	Auto-incrementing — SQL Server

CORE CONSTRAINTS

CONSTRAINT	DESCRIPTION
`PRIMARY KEY`	Unique row identifier — not null
`NOT NULL`	Column must always have a value
`UNIQUE`	All values must be different
`DEFAULT value`	Value when none is provided
`CHECK (condition)`	Value must satisfy condition
`FOREIGN KEY ... REFERENCES`	Must exist in referenced table

FOREIGN KEY & ALTER

SYNTAX	DESCRIPTION
`FOREIGN KEY (col) REFERENCES t(col)`	Table level foreign key
`ON DELETE CASCADE`	Delete children with parent
`ON DELETE SET NULL`	Set null when parent deleted
`ON UPDATE CASCADE`	Update children with parent
`ALTER TABLE t ADD CONSTRAINT name FK...`	Add named foreign key
`ALTER TABLE t DROP CONSTRAINT name`	Remove a named constraint

⚠ COMMON MISTAKES

- No primary key — tables without one are harder to manage and query
- `ON DELETE CASCADE` without thinking it through — parent delete silently wipes children
- Unnamed constraints — impossible to drop by name later
- Forgetting `CHECK` constraints are not enforced in older MySQL versions

✓ TIPS & BEST PRACTICES

- Name every constraint — `CONSTRAINT fk_orders_customer FOREIGN KEY...`
- Think carefully before using `ON DELETE CASCADE` — prefer `SET NULL` or restrict
- Add constraints after data load for performance, then validate
- Use `UNIQUE` constraints on business keys like email or order number

QUICK SUMMARY

- Constraints enforce data rules at the database level
- Always name constraints for easy management
- `CASCADE` is powerful but dangerous — use deliberately
- Every table should have a `PRIMARY KEY`

Views

CREATE VIEW · DROP VIEW · updatable views · WITH CHECK OPTION · materialized views

DESCRIPTION

A view is a saved `SELECT` query stored in the database under a name and queried like a regular table. Views do not store data — they execute the underlying query each time they are referenced. They are useful for simplifying complex queries, restricting column access, and presenting a consistent interface even if the underlying table structure changes. A *materialized view* stores the result physically for faster reads but requires periodic refresh.

NOTES

- Views execute their query every access — complex views on large tables can be slow
- Dropping a table that a view depends on will break the view
- Views cannot accept parameters — use a stored procedure or CTE instead
- Materialized views must be explicitly refreshed — data may be stale

EXAMPLE

```sql
-- Create or replace a view
CREATE OR REPLACE VIEW engineering_active AS
SELECT id, first_name, last_name, email, salary
FROM   employees
WHERE  department = 'Engineering'
  AND  status = 'active'

-- Query the view like a table
SELECT * FROM engineering_active
WHERE  salary > 80000
ORDER BY last_name

-- Drop the view
DROP VIEW IF EXISTS engineering_active
```

CODE EXPLAINED

- → `CREATE OR REPLACE` — create or update without dropping first
- → View restricts columns — `id, first_name...` not the full table
- → View restricts rows — only Engineering + active employees visible
- → `SELECT * FROM engineering_active` — used like any table
- → Additional `WHERE` filters view results further

UPDATABLE VIEW REQUIREMENTS

REQUIREMENT	DETAIL
`Single table`	No JOINs in the view definition
`No DISTINCT`	Must not deduplicate rows
`No GROUP BY`	Must not collapse rows
`No aggregates`	No SUM, COUNT, etc. in SELECT
`Includes PK`	Primary key column must be present

⚠ COMMON MISTAKES

- Dropping a base table without dropping dependent views first
- Expecting a view to cache results — it re-runs the query every time
- Trying to pass parameters to a view — use a stored procedure instead
- Forgetting to refresh a materialized view — stale data can cause issues

SYNTAX

SYNTAX	DESCRIPTION
`CREATE VIEW name AS SELECT...`	Create a new view
`CREATE OR REPLACE VIEW name AS...`	Create or update existing
`DROP VIEW name`	Delete a view
`DROP VIEW IF EXISTS name`	Delete only if it exists
`SELECT * FROM view_name`	Query like a table
`... WITH CHECK OPTION`	Prevent inserts/updates invisible to view

✓ TIPS & BEST PRACTICES

- Use views to expose only specific columns/rows to users without granting full table access
- Use `CREATE OR REPLACE` instead of `DROP + CREATE` to avoid permission issues
- Use materialized views for expensive queries that don't need real-time data
- Use `WITH CHECK OPTION` to enforce view filter consistency on inserts/updates

MATERIALIZED VIEWS

SYNTAX	DESCRIPTION
`CREATE MATERIALIZED VIEW name AS SELECT...`	Stores result physically — PostgreSQL / Oracle
`REFRESH MATERIALIZED VIEW name`	Update stored result — PostgreSQL
`REFRESH MATERIALIZED VIEW CONCURRENTLY name`	Refresh without locking reads

QUICK SUMMARY

- Saved SELECT query — queried like a table
- No data stored — re-executes query each access
- Updatable only under strict conditions
- Materialized views store data — faster but need refresh

Transactions

BEGIN · COMMIT · ROLLBACK · SAVEPOINT · isolation levels · auto-commit

DESCRIPTION

A transaction is a group of SQL statements executed as a single unit — either all succeed or all fail together. Transactions protect data integrity by ensuring that partial updates never leave the database in an inconsistent state. Most databases run in auto-commit mode by default — each statement is its own transaction unless you explicitly `BEGIN`. Always `COMMIT` or `ROLLBACK` — leaving a transaction open locks rows and blocks other queries.

NOTES

- Always `COMMIT` or `ROLLBACK` — open transactions lock rows
- `TRUNCATE` cannot be rolled back in some databases — use `DELETE` inside a transaction if rollback is needed
- Nested transactions not supported in most databases — use `SAVEPOINT`
- Long-running transactions increase deadlock risk — keep them short

EXAMPLE — BANK TRANSFER

```sql
BEGIN;

-- Deduct from sender
UPDATE accounts
SET    balance = balance - 500
WHERE  id = 101;
-- Add to receiver
UPDATE accounts
SET    balance = balance + 500
WHERE  id = 202;
-- Verify both accounts look correct
SELECT id, balance FROM accounts WHERE id IN (101, 202);

COMMIT;  -- or ROLLBACK if something went wrong
```

CODE EXPLAINED

- → `BEGIN` — starts the transaction block
- → Both UPDATEs are part of the same atomic unit
- → If either fails, neither is committed
- → `SELECT` — verify state before committing
- → `COMMIT` — makes both changes permanent
- → `ROLLBACK` — undoes everything since BEGIN

SETTING ISOLATION

SYNTAX	DESCRIPTION
`SET TRANSACTION ISOLATION LEVEL READ COMMITTED`	Set for current transaction
`SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ`	Set for session — MySQL

CORE SYNTAX

SYNTAX	DESCRIPTION
`BEGIN`	Start transaction (START TRANSACTION in MySQL)
`COMMIT`	Save all changes permanently
`ROLLBACK`	Undo all changes since BEGIN
`SAVEPOINT name`	Create named checkpoint within transaction
`ROLLBACK TO name`	Partial undo to savepoint
`RELEASE SAVEPOINT name`	Remove a savepoint

ISOLATION LEVELS

LEVEL	DESCRIPTION
`READ UNCOMMITTED`	Can read uncommitted changes — fastest, least safe
`READ COMMITTED`	Only reads committed data — most DB default
`REPEATABLE READ`	Same row returns same result within transaction
`SERIALIZABLE`	Strictest — transactions run as if sequential

⚠ COMMON MISTAKES

- Leaving a transaction open — locks rows and blocks other queries indefinitely
- Using `TRUNCATE` inside a transaction expecting it to roll back — may not work
- Trying to nest transactions — use `SAVEPOINT` instead
- Long transactions on busy tables — greatly increases deadlock risk

✓ TIPS & BEST PRACTICES

- Keep transactions as short as possible — open, modify, commit
- Use `SAVEPOINT` for partial rollback in complex multi-step operations
- Wrap risky `UPDATE` / `DELETE` in a transaction to test before committing
- Use `READ COMMITTED` isolation for most OLTP workloads

QUICK SUMMARY

- All or nothing — either all statements succeed or none do
- Always end with `COMMIT` or `ROLLBACK`
- `SAVEPOINT` enables partial rollback
- Keep transactions short to reduce lock contention

SQL · Sheet 20

Stored Procedures

CREATE PROCEDURE · IN / OUT / INOUT · IF · WHILE · DECLARE · CALL

DESCRIPTION

A stored procedure is a named, reusable block of SQL saved in the database that can accept parameters, execute multiple statements, and contain conditional logic and loops. Unlike a view, a stored procedure can modify data, manage transactions, and return multiple result sets. Syntax varies more between databases than almost any other SQL feature — the examples below lean toward MySQL. Code written for MySQL rarely runs unchanged in PostgreSQL or SQL Server.

NOTES

- Stored procedures are highly database-specific
- Use `DELIMITER $$` in MySQL — prevents `;` from ending the definition early
- Procedures can call other procedures
- Overuse makes application logic hard to version control and test

EXAMPLE — MYSQL

```
DELIMITER $$

CREATE PROCEDURE give_raise(
    IN  dept_name VARCHAR(50),
    IN  raise_pct DECIMAL(5,2)
)
BEGIN
    DECLARE affected INT DEFAULT 0;

    UPDATE employees
    SET    salary = salary * (1 + raise_pct / 100)
    WHERE  department = dept_name
      AND  status = 'active';

    SET affected = ROW_COUNT();
    SELECT CONCAT(affected, ' employees updated') AS result;
END$$

DELIMITER ;

CALL give_raise('Engineering', 10.00);
```

CODE EXPLAINED

- → `DELIMITER $$` — change delimiter so ; inside doesn't end the definition
- → `IN dept_name` — input parameter passed by caller
- → `DECLARE affected INT` — local variable with default
- → `UPDATE` — procedure modifies data directly
- → `ROW_COUNT()` — count of rows affected by last statement
- → `CALL give_raise(...)` — execute the procedure

PARAMETERS

TYPE	DESCRIPTION
`IN param datatype`	Input — passed in, cannot be changed
`OUT param datatype`	Output — returned to the caller
`INOUT param datatype`	Both — passed in and returned

CORE SYNTAX

SYNTAX	DESCRIPTION
`CREATE PROCEDURE name() BEGIN...END`	Create — MySQL
`CREATE OR REPLACE PROCEDURE name() AS $$...$$`	Create or replace — PostgreSQL
`CALL procedure_name(arg1, arg2)`	Execute with arguments
`DROP PROCEDURE IF EXISTS name`	Delete only if it exists

CONTROL FLOW

SYNTAX	DESCRIPTION
`IF cond THEN...END IF`	Basic conditional
`IF cond THEN...ELSE...END IF`	If / else
`WHILE cond DO...END WHILE`	While loop — MySQL
`DECLARE var datatype DEFAULT val`	Local variable
`SET var = value`	Assign to variable

⚠ COMMON MISTAKES

- Forgetting `DELIMITER $$` in MySQL — `;` inside the body ends the definition
- Writing MySQL procedures and expecting them to run in PostgreSQL
- Embedding too much business logic in procedures — hard to test and version control
- Not handling errors — unhandled exceptions leave transactions open

✓ TIPS & BEST PRACTICES

- Use procedures for database-level operations (bulk updates, complex transforms)
- Keep procedures focused — one procedure per logical operation
- Always reset `DELIMITER ;` after creating a MySQL procedure
- Add error handling blocks to prevent silent failures

QUICK SUMMARY

- Named, reusable SQL block stored in the database
- Accepts IN, OUT, and INOUT parameters
- Syntax is highly database-specific
- Use `CALL` to execute

SQL · Sheet 21

Triggers

BEFORE / AFTER · INSERT / UPDATE / DELETE · OLD and NEW references · audit logging

DESCRIPTION

A trigger is a stored procedure that automatically executes in response to a specific event on a table — an `INSERT`, `UPDATE`, or `DELETE`. Triggers fire either before or after the event and have access to the old and new row values through `OLD` and `NEW`. They are useful for audit logging, enforcing business rules, and auto-updating related data. Like stored procedures, trigger syntax varies significantly between databases.

NOTES

- Triggers fire automatically — invisible to the application, hard to debug
- Avoid triggers that modify the same table — can cause infinite loops
- Triggers add overhead to every affected operation
- PostgreSQL triggers require a separate trigger function in PL/pgSQL

EXAMPLE — SALARY CHANGE AUDIT LOG

```
DELIMITER $$

CREATE TRIGGER log_salary_change
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF OLD.salary != NEW.salary THEN
        INSERT INTO salary_log
            (employee_id, old_salary, new_salary, changed_by)
        VALUES
            (OLD.id, OLD.salary, NEW.salary, CURRENT_USER());
    END IF;
END$$

DELIMITER ;
```

CODE EXPLAINED

- → `AFTER UPDATE` — fires after each row is updated
- → `FOR EACH ROW` — runs once per affected row
- → `OLD.salary` — the value before the update
- → `NEW.salary` — the value after the update
- → `IF OLD != NEW` — only log when salary actually changed
- → `CURRENT_USER()` — captures who made the change

COMMON USES

USE CASE	DESCRIPTION
`Audit logging`	Record who changed what and when
`Auto timestamps`	Set updated_at = NOW() on every update
`Business rules`	Reject changes that violate conditions
`Sync related tables`	Auto-update summary or archive tables
`Soft delete`	Intercept DELETE — mark inactive instead

⚠ COMMON MISTAKES

- Trigger modifying its own table — can create an infinite loop
- Forgetting triggers fire silently — application won't know unless it queries the log
- Overly complex trigger logic — slows every affected INSERT / UPDATE / DELETE
- Not testing trigger behavior in a transaction before deploying

CORE SYNTAX

SYNTAX	DESCRIPTION
`CREATE TRIGGER name BEFORE INSERT ON t FOR EACH ROW...`	Fire before each insert — MySQL
`CREATE TRIGGER name AFTER UPDATE ON t FOR EACH ROW...`	Fire after each update
`CREATE TRIGGER name AFTER DELETE ON t FOR EACH ROW...`	Fire after each delete
`CREATE OR REPLACE TRIGGER name...`	Create or replace — PostgreSQL / Oracle
`DROP TRIGGER IF EXISTS name`	Delete only if it exists

✓ TIPS & BEST PRACTICES

- Keep trigger logic minimal — complex logic belongs in the application
- Always check `OLD != NEW` before logging to avoid noise
- Use `BEFORE` triggers to validate or transform data before it's written
- Document triggers clearly — they are invisible to most application developers

TIMING & OLD / NEW

KEYWORD	DESCRIPTION
`BEFORE`	Fires before — can modify NEW values
`AFTER`	Fires after — cannot modify the row
`INSTEAD OF`	Replaces operation — used on views
`NEW.col`	New value — INSERT and UPDATE triggers
`OLD.col`	Previous value — UPDATE and DELETE triggers
`NEW.col = value`	Modify incoming value in BEFORE trigger

QUICK SUMMARY

- Auto-execute on INSERT, UPDATE, or DELETE
- `OLD` = before, `NEW` = after
- `BEFORE` can modify data; `AFTER` cannot
- Keep logic simple — they fire on every affected row

SQL · Sheet 22

Indexes

CREATE INDEX · composite indexes · UNIQUE index · index types · when to index · EXPLAIN

DESCRIPTION

An index speeds up data retrieval by allowing the database to find rows without scanning the entire table — like a book's index pointing to the right page. Indexes come with a trade-off: they speed up reads but slow down writes (`INSERT`, `UPDATE`, `DELETE`) because each index must be updated alongside the data. Primary keys are automatically indexed in all major databases.

NOTES

- Primary keys are automatically indexed — no need to create one manually
- Composite indexes only help when the query filters on the leftmost column(s) first
- The query optimizer decides whether to use an index — it may skip it if the table is small
- Too many indexes slow down writes — audit and drop unused indexes regularly

EXAMPLE

```
-- Index for a common lookup
CREATE INDEX idx_employees_department
ON employees (department);
-- Composite index for filtering and sorting together
CREATE INDEX idx_employees_dept_salary
ON employees (department, salary DESC);
-- Unique index to enforce a business rule
CREATE UNIQUE INDEX idx_employees_email
ON employees (email);
-- Check if the index is actually being used
EXPLAIN SELECT * FROM employees
WHERE    department = 'Engineering'
ORDER BY salary DESC;
```

CODE EXPLAINED

- → `idx_employees_department` — named for easy identification and drop
- → Composite `(department, salary DESC)` — covers both filter and sort
- → `CREATE UNIQUE INDEX` — index that also enforces uniqueness
- → `EXPLAIN` — shows whether the optimizer uses the index or does a full scan

WHEN TO INDEX / NOT INDEX

GOOD CANDIDATE	POOR CANDIDATE
WHERE clause columns	Small tables (full scan faster)
JOIN ON columns	Low cardinality (BOOLEAN, status)
ORDER BY columns	Frequently updated columns
Foreign key columns	Columns rarely used in queries
High cardinality columns	Too many indexes — slows writes

SYNTAX

SYNTAX	DESCRIPTION
`CREATE INDEX name ON t (col)`	Basic index — one column
`CREATE INDEX name ON t (col1, col2)`	Composite — multiple columns
`CREATE UNIQUE INDEX name ON t (col)`	Unique — enforces no duplicates
`CREATE INDEX name ON t (col DESC)`	Descending sort order
`DROP INDEX name ON t`	Delete an index — MySQL
`DROP INDEX IF EXISTS name`	Delete only if it exists

INDEX TYPES & CHECKING USAGE

TYPE / COMMAND	DESCRIPTION
`B-Tree`	Default — equality, range, sort
`Hash`	Fast equality only — no range
`Full-text (FULLTEXT)`	Searching within text content
`EXPLAIN SELECT...`	Show query execution plan
`EXPLAIN ANALYZE SELECT...`	Plan with actual stats — PostgreSQL
`SHOW INDEX FROM t`	List all indexes on table — MySQL

⚠ COMMON MISTAKES

- Indexing every column — more indexes = slower writes on every change
- Composite index column order wrong — optimizer ignores it if leftmost column not in query
- Not indexing foreign key columns — join performance suffers significantly
- Assuming an index will always be used — the optimizer may choose a full scan

✓ TIPS & BEST PRACTICES

- Always name indexes clearly — `idx_tablename_column`
- Use `EXPLAIN` to verify that your index is actually being used
- Index foreign key columns — they are often not auto-indexed
- Audit indexes periodically — drop any that are unused to reduce write overhead

QUICK SUMMARY

- Speeds reads, slows writes — use deliberately
- Primary keys are auto-indexed
- Composite indexes: leftmost column matters
- Use `EXPLAIN` to verify index usage